

Moringa Backend & DB Fast-Track Study Guide

A concise, high-yield summary of Flask, FastAPI, SQLite & PostgreSQL for night review

This guide brings together all essential concepts, configurations, and code snippets covered from Flask and FastAPI introduction up to the current Relational Databases (SQLite & PostgreSQL) lessons. Keep this handy for quick reference and class prep.

1. FLASK OVERVIEW & SERVER-SIDE RENDERING (SSR)

Flask is a minimalist Python WSGI microframework. It provides lightweight routing, request handling, and template rendering without imposing rigid database or architecture choices.

Environment Setup & Virtual Environments

- > pipenv shell Activates the project virtual environment isolating installed dependencies.
- > pipenv install flask Installs the Flask framework into your Pipfile.
- > Running Server Run 'python app.py' (with app.run(debug=True)) or 'flask run' in your terminal.

Flask Core App Template (app.py)

```
from flask import Flask, render_template, request, jsonify

app = Flask(__name__)

@app.route('/')
def home():
    users = [{'id': 1, 'name': 'Daniel'}, {'id': 2, 'name': 'Arthur'}]
    return render_template('home.html', users=users, title='Home Page')

@app.route('/api/user/<int:user_id>', methods=['GET'])
def get_user(user_id):
    return jsonify({'status': 'success', 'user_id': user_id})

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

Jinja2 Server-Side Rendering (SSR)

Flask automatically looks for HTML templates in a folder named 'templates/' and static assets (CSS, JS, images) in 'static/'.

- > {{ variable }} Interpolates Python data into HTML (e.g. {{ title }}).
- > {% for item in list %} ... {% endfor %} Renders repeated elements dynamically.
- > {% if condition %} ... {% else %} ... {% endif %} Conditional block rendering.

2. FASTAPI: MODERN ASYNCHRONOUS APIS

FastAPI is an ASGI framework built on top of Starlette and Pydantic. It is designed for high performance, automatic data validation, and automated interactive OpenAPI documentation.

Setup & Running FastAPI

- > Installation: `pipenv install "fastapi[standard]"` (installs FastAPI, Uvicorn, and standard tooling).
- > Development Server: `pipenv run fastapi dev app.py` (launches hot-reloading ASGI dev server).
- > Interactive Docs: Open your browser to `http://127.0.0.1:8000/docs` (Swagger UI) or `/redoc`.

FastAPI Core Application & Validation (app.py)

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Optional

app = FastAPI(title='Moringa API')

class StudentSchema(BaseModel):
    name: str
    email: str
    phone: Optional[int] = None
    marks: float

@app.get('/')
def root():
    return {'message': 'Hello world'}

@app.post('/students', status_code=201)
def create_student(student: StudentSchema):
    # student is automatically parsed, type-checked, and validated
    return {'status': 'created', 'data': student.model_dump()}
```

Key Contrast: Flask vs. FastAPI

Feature	Flask	FastAPI
Server Standard	WSGI (Synchronous by default)	ASGI (Native async / await support)
Data Validation	Manual (request.json, custom checks)	Automatic via Pydantic type schemas
API Documentation	Manual or third-party extensions	Auto-generated at /docs (Swagger) and /redoc
Primary Use Case	SSR with Jinja2 templates, full-stack apps	High-performance REST APIs & microservices

3. RELATIONAL DATABASES & SQL (THE CURRENT CLASS FOCUS)

A Database Management System (DBMS) stores and retrieves structured data. Relational DBs (RDBMS) use tables with rows and columns, enforce schemas, and use Structured Query Language (SQL).

Relational (SQL) vs. Non-Relational (NoSQL)

- > Relational (RDBMS): Strict schema, ACID compliance, relational queries (JOINS). Examples: SQLite, PostgreSQL, MySQL.
- > Non-Relational (NoSQL): Flexible/dynamic schema, documents/key-value. Examples: MongoDB, Redis, DynamoDB.

SQL Syntax Rules

- > Case Insensitivity: Keywords like SELECT, CREATE, DROP are case-insensitive (uppercase is convention).
- > Termination: SQL statements must terminate with a semicolon (;).
- > Atomicity: Database transactions are atomic: either all operations succeed or all rollback.

SQLite vs. PostgreSQL: Side-by-Side Comparison

Concept	SQLite	PostgreSQL
Architecture	Embedded file on disk (db.db), no server	Client-Server architecture (Port 5432)
Auto Primary Key	INTEGER PRIMARY KEY AUTOINCREMENT	SERIAL PRIMARY KEY / BIGSERIAL PRIMARY KEY
Text / String	TEXT	VARCHAR(length), TEXT, CHAR(n)
Booleans	INTEGER (0 for false, 1 for true)	BOOLEAN (native TRUE / FALSE)
Advanced Types	Stores JSON as TEXT	Native JSON, JSONB, TIMESTAMPTZ, UUID

SQLite Code Covered in Class

SQLite Table Creation (DB/Sqlite/intro_create_tables.sql)

```
-- 1. Create a basic test table
CREATE TABLE test_table (
    id INTEGER PRIMARY KEY
);

-- 2. Drop table example
DROP TABLE test_table;

-- 3. Student table using SQLite data types
CREATE TABLE student (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT,
    phone INTEGER,
    is_married INTEGER,    -- SQLite stores booleans as 0 or 1
    marks REAL,           -- Real numbers (floats)
    dob DATE
);
```

PostgreSQL Code & Column Constraints Covered in Class

PostgreSQL Schema & Constraints (DB/Postgres/pg_intro_creating_tables.sql)

```
-- 1. Student table with PostgreSQL native types
CREATE TABLE student (
    id SERIAL PRIMARY KEY,
    name VARCHAR(20),
    email VARCHAR(50),
    phone INTEGER,
    is_married BOOLEAN,           -- Native boolean
    marks NUMERIC,               -- Exact decimal representation
    spirit_animal JSON,           -- Native JSON data
    dob DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    created_at_timezone TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

-- 2. Inventory table demonstrating data integrity constraints
CREATE TABLE inventory (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL,    -- Must have a name
    barcode INTEGER NOT NULL UNIQUE, -- Barcode must be unique
    product_code VARCHAR(100) UNIQUE, -- Distinct nullable value
    buying_price INTEGER NOT NULL CONSTRAINT buying_price_must_be_greater_than_0 CHECK (buying_price > 0),
    selling_price INTEGER NOT NULL CHECK (selling_price > 0),
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

4. HOW DATABASES ARE CONNECTED & TOOLING GUIDE

In the current lessons, the instructor connected directly using graphical database inspection clients. Here is exactly how to set them up:

Connecting in VS Code (Database Client Extension)

- > 1. Install Extension: In VS Code Extensions marketplace (Cmd+Shift+X), install 'Database Client' (by cweijan) or 'SQLTools'.
- > 2. Connect to SQLite: Click '+ Add Connection' -> Select SQLite -> Point file path to 'DB/Sqlite/db.db' -> Connect.
- > 3. Connect to PostgreSQL: Click '+ Add Connection' -> Select PostgreSQL -> Host: 127.0.0.1, Port: 5432, User: postgres, Password: <your-pass>.
- > 4. Running SQL: Right-click your database connection -> 'New Query' (Query Console) -> Write and execute SQL queries.

Command Line Cheat Sheet

Action	Command
Activate Virtualenv	pipenv shell
Run Flask App	python app.py or flask run
Run FastAPI App	pipenv run fastapi dev app.py
Open SQLite DB in CLI	sqlite3 DB/Sqlite/db.db
SQLite View Schema	.schema
SQLite List Tables	.tables
Exit SQLite CLI	.exit

Your Night Study Checklist

- > [x] SQLite db.db Initialized: Your 'DB/Sqlite/db.db' is already loaded with 'test_table' and 'student' tables.
- > [] Connect Database Client: Open VS Code, install Database Client extension, and inspect 'db.db'.
- > [] Review SQL Differences: Make sure you know 'SERIAL' vs 'INTEGER PRIMARY KEY AUTOINCREMENT', and 'VARCHAR' vs 'TEXT'.
- > [] Review Constraint Syntax: Know 'NOT NULL', 'UNIQUE', and 'CHECK (price > 0)'.